

From Proof-Order Graphs to Checked Proof Reuse

A source-pinned comparison and joint research plan for ProofNet

Research design note for the ProofNet collaboration

30 September 2026

Abstract

This note compares the current ProofNet-IR and proof-graphs repositories with the available Boundary-Lens prototype. The repositories already supply a verified restricted MLL library, Lean graph extraction, structural identities, coupled-goal search, and negative empirical results that narrow the case for further order-based pruning. We recommend one integrated program: retain the formal MLL layer, extend the existing Lean engine with typed transaction-aware boundaries, and test completed proof-fragment transport against a conventional typed cache. We separate standard substitution principles from proposed engineering and distinguish final kernel replay from observed tactic effects. New executable results here are a Python audit of the supplied prototype and finite-model regression tests, not fresh Lean or neural-prover experiments. Source pins, explicit evidence levels, falsifiable experiments, and implementation prompts make the proposed collaboration auditable.

Contents

Reading this note	2
What the two repositories change about your part	3
The honest comparison	3
What your friend has actually contributed	3
Where your contribution still has value	4
Findings from this review	5
What to retire from the old pitch	5
Proposed joint position	5
Revised joint methodology: from graph shape to checked reuse	6
Separate four objects	6
Boundary and event definitions	6
Evidence hierarchy	7
The hashing limitation: an elementary but decisive result	7
Conditional local independence, not tactic-name independence	7
The recommended new algorithm: closed-fragment transport	8
Three instructive examples	9
Interface and integration	9
Scope of novelty	11
Experiment and implementation plan	11
Work package 1: reproduce and audit the actual baseline	11
Work package 2: a fail-closed typed boundary sidecar	11
Work package 3: closed-fragment cache with a graph ablation	12
Work package 4: held-out mathematical pilot	12
Controls that matter	12
Go / no-go gates	12
Contribution and collaboration proposal	13
Your part, stated without inflating it	13

Proposed ownership, subject to your agreement	13
Proposed sequence of pull requests	13
Suggested message to your friend	14
What was and was not executed	14
Actually executed here	14
Not executed here	14
Interpretation limits	14
Source register	14

Reading this note

The conclusions compare inspected artifacts, not the collaborators' personal abilities or complete contribution histories. Numerical repository results below are first-party reports, not new replications. The review could read selected current GitHub files, but the container had no Lean/Lake and could not clone the repositories. The local user-artifact audit and the reference Python tests were executed. All new live-graph and proof-reuse examples are design fixtures.

The package contains 29 passing Python representation/schema tests. Those tests exercise a deliberately restricted expression model. They neither typecheck Lean terms nor prove that a proposed normalization preserves all of Lean's operational behavior.

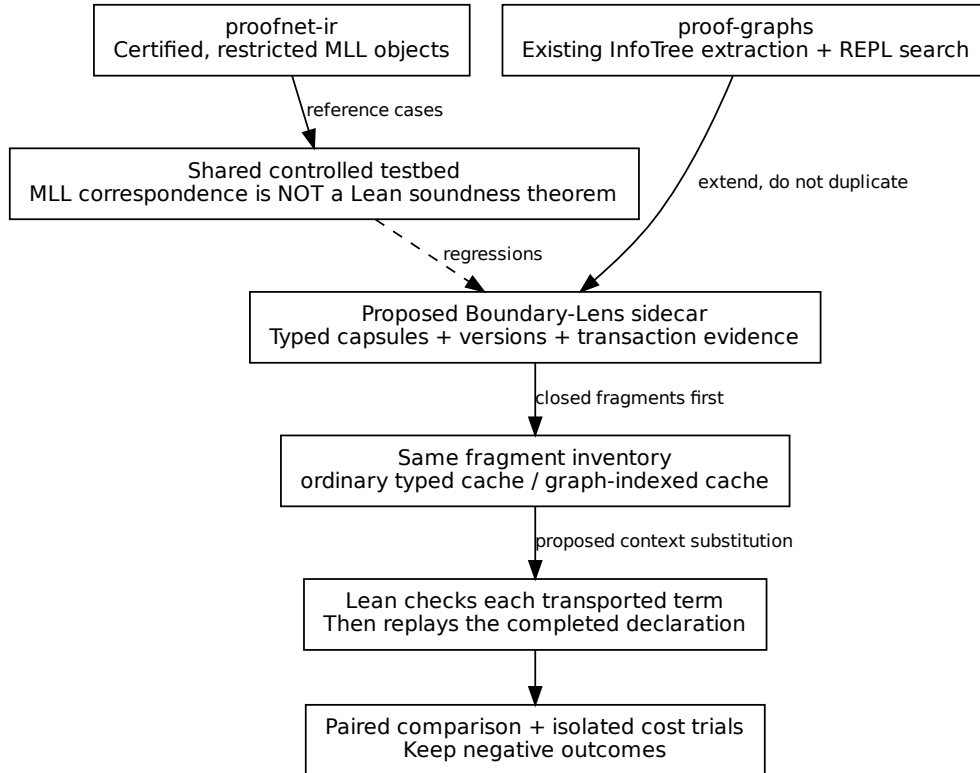


Figure 1: One integration, not three competing extractors. The MLL reference arrows express reuse of test cases and concepts, not a soundness reduction for dependent-type Lean proofs.

What the two repositories change about your part

Decision: integrate your work into **proof-graphs**; do not build another parallel Lean graph extractor. Retain **proofnet-ir** as the formally verified MLL reference layer. Recast Boundary-Lens as a **typed, transaction-aware evidence and proof-reuse interface**, evaluated against your friend’s existing structural-state and coupled-group search baselines.

This is a recommendation about research direction, not a judgment about authorship or personal ability. The code is treated as your friend’s contribution because that is how you described it. The repositories alone do not establish who contributed every idea, nor how publication credit should be divided.

The honest comparison

At the inspected commits, your friend’s work is ahead of the last Boundary-Lens artifact in both implementation and experimental maturity. Your earlier design remains useful, but much of its originally proposed engineering is now overlapping work. The right response is to reuse that foundation rather than defend an obsolete novelty claim. [R1–R8, U1]

Dimension	proofnet-ir	proof-graphs	Your last available Boundary-Lens package
Core object	Actual proof-net certificates for a restricted MLL calculus	Extracted step-dependency graphs and live search graphs	Goal-boundary snapshots and proposed typed graph updates
Trust basis	Lean-formalized checker/sequentialization results in that calculus	Executed Lean traces, structural checks, final proof replay	Included controlled Lean logs and custom frontier checks
Data scale	Formal-library and MLL experiment artifacts	7,285 corrected step graphs, plus search artifacts	Five controlled theorems, 15 success rows and five failure rows in the inspected ZIP
Identity	Canonical identity for the specified MLL representation	Lean-expression keys, renaming, and coupled groups	Raw goal transport; schema and feature separation still incomplete
Search evidence	Registered restricted-logic experiments	Several registered Lean search comparisons	No demonstrated incremental search improvement
Best role in joint project	Logical reference and certified restricted testbed	Shared implementation and empirical baseline	Proposed evidence-quality, transport, and checked-reuse layer

The 7,285 graphs are not 7,285 fully annotated open-boundary traces. The exported rows contain step kinds, source lines, edges, and graph statistics; expression-rich state snapshots cannot be reconstructed from that dataset alone. A sidecar enrichment must return to raw extraction or re-elaborate a source subset. [R2, R7, R8, R17]

What your friend has actually contributed

A. A genuine formal proof-net library

proofnet-ir establishes substantial guarantees for **unit-free, cut-free multiplicative linear logic**: a correctness checker, conversion from derivations, sequentialization of accepted certificates, and canonical identity. Its rolling result includes a stated quadratic symbolic-operation bound, not a general linear-time guarantee or a bound on arbitrary Lean proof search. This is a formal-methods contribution in its own right. [R1]

It is not simply a backend that can certify any Lean dependency graph. Lean’s dependent type theory and tactic meta-state are not MLL; a bridge requires a separate definition and theorem. Shared terminology is not such a bridge.

B. An actual extraction and search program

proof-graphs already traverses Lean InfoTrees, derives dependency graphs, drives a live Lean REPL, compares state and goal searches, normalizes identities structurally, and keeps metavariable-coupled goals together. It also reconstructs proof scripts and checks completed candidates from the statement. Those are implementations, not only plans. [R5–R8]

Consequently, “we use real Lean states,” “we preserve metavariable identity,” “we group coupled goals,” and “we reuse solved goals” are no longer distinctive next contributions for your branch.

C. Useful negative evidence

The most valuable change is empirical. The repositories distinguish the number of possible orderings of a completed proof graph from the order redundancy actually encountered by a search policy. Default first-goal scheduling can suppress the latter without eliminating the former. [R2–R4]

In search-v0.8, the registered comparable set contains **158 tasks, with three draw sets per task**:

Search	Set 0	Set 1	Set 2	Total / 474 task-sets
Whole states	38	37	41	116
Independent goals	40	39	42	121
Coupled goal groups	40	39	43	122

None of the three registered pairwise comparisons passed the Holm-adjusted decision rule. These are neither 474 independent theorems nor proof of equivalence between algorithms. They are evidence against expecting a large automatic benefit from replacing state search by this graph search in the tested setting. [R3]

The latest correction explicitly withdraws a simple runtime-cost advantage: searches share model draws and run in a rotating order, so later searches can inherit cached work. Both the pooled timing and the “ran first” subsets are inadequate as a clean isolated latency comparison. [R3]

A further update matters in **proofnet-ir**: with the thinking condition in representation-v0.3, 35 nets, 32 labelled derivations, and 30 positional derivations verify out of 90 tasks; no pairwise format difference is significant after the adjustment. Without thinking the counts are 11, 0, 0. An earlier headline that nets reliably outperform derivations is too broad. [R9]

Where your contribution still has value

Immediate, defensible value: independent semantic audit

The existing handoff identifies unaudited failure combinators and normalization concerns. A genuinely independent reconstruction and adversarial Lean test suite is useful work, particularly because structural invariants can pass when an extractor consistently assigns the wrong interpretation. [R4]

Your Boundary-Lens perspective is valuable here: it asks what each edge means, what state it belongs to, and what evidence permits reuse. Convert that perspective into executable checks rather than another layer of terminology.

Medium-term value: a shared semantic interface

The offline extractor records goal identifiers and tactic-tree structure, while the live search exporter records instantiated expressions. A joint interface can connect these without treating them as identical evidence: versioned goal capsules, transaction/branch identities, assignment deltas, term/local provenance, and an explicit record of incomplete dependency coverage. [R5–R7]

This is a proposed integration contribution. It is not yet implemented in Lean in this deliverable, and it should not be advertised as a new proof theory by naming it a lens.

High-upside, uncertain value: transportable proof fragments

The new experiment should target a narrower question:

Can a boundary-aware index find and replay already checked proof fragments in new compatible contexts, avoiding repeated tactic work beyond structural-state deduplication and ordinary proof-term caching?

Existing search already shares proofs for matching goal keys. The possible extension is **typed context transport**: reuse when irrelevant local assumptions differ, when required local evidence can be mapped to a different context, or when the same subargument appears inside different surrounding proofs. Start with closed fragments; require Lean to check each transported application. Graph indexing must beat a non-graph cache receiving the same fragment pool before graph-specific value is claimed. [R6; proposed extension]

This has no guaranteed payoff. It may turn out that ordinary premise retrieval or a typed cache captures almost everything. That outcome would still determine which part of your representation is worth keeping.

Findings from this review

A fresh Python audit of the available v0.7.1 ZIP found 15 success records and five failure records, no typed edges in any success delta, no explicit `rho_before` in those rows, and 15/15 failures against the shipped trace schema. That refines earlier conversation counts: use the inspected artifact’s five failures, not a remembered six. No Lean execution was rerun here. [U1]

Reviewing `goal_identity.py` also revealed a concrete **testable hazard**: metavariable renaming uses regular expressions over the complete serialized expression string, including quoted string literals. Thus a literal containing a metavariable-looking token can be altered as though it were a genuine metavariable. This package reproduces that mechanism in small Python fixtures and shows how a tagged AST avoids it. This is not evidence that a particular published Lean proof was wrong or that the empirical conclusions are invalid. It is a regression to verify in actual Lean. [R5]

Other bounded audit targets are an `h?` fallback for unexported free variables, omitted implementation-detail locals, grouping by directly visible term-metavariable mentions, and a printed carried-goal check in the otherwise structural-key search. These deserve conservative handling and tests. They are not a blanket claim that the entire engine is unsound. Final proof replay protects theorem acceptance, but unsafe merging can still discard solvable branches or distort measurements. [R5, R6]

What to retire from the old pitch

Retire “factorial proof-order reduction implies factorial Lean search savings.” Retire “state plus graph hashing safely prunes more than faithful state hashing.” Retire “only the focused goal can change.” Retire “every successful tactic or recorded dependency edge is kernel-certified.” Keep the mathematical motivation, but distinguish formal theorems, executed observations, derived annotations, and speculative proposals.

Do not lead with a PFR or condensed-mathematics diagram whose states were not extracted. Use those areas later as held-out semantic stress tests, not as decorative evidence of scale.

Proposed joint position

A suitable joint research question is:

Which proof-graph abstractions remain useful once ordinary goal scheduling and structural-state sharing have already removed the easy redundancy, and what evidence is needed to reuse a proof fragment safely?

A suitable description of your part is:

I am developing the typed boundary and proof-transport layer that turns extracted search graphs into auditable, reusable proof fragments, and testing its incremental value against the project’s existing state and graph baselines.

That is stronger than another general graph extractor because it connects directly to the evidence your friend already collected. It is also honest about what still has to be implemented and measured.

Revised joint methodology: from graph shape to checked reuse

Status: proposed method and executable Python design tests. No new Lean soundness theorem, neural-prover result, or frontier extraction is claimed here. Mathematical propositions below have explicit hypotheses; the reference implementation is a tagged-expression model, not Lean’s meta-state.

Separate four objects

Use four related but distinct representations.

1. **MLL certificate:** the object checked by `proofnet-ir`. Its correctness results apply only to its specified logic. [R1]
2. **Offline derivation graph:** the corrected `proof-graphs` step DAG. It is useful for structural analysis and supervised metadata. It is not an online state. [R7, R8]
3. **Live boundary/event graph:** accepted and attempted Lean transitions with source, state, and evidence identities. This is the proposed Boundary-Lens integration.
4. **Reuse index:** a retrieval structure for completed proof fragments. A match proposes an application; Lean decides whether that application is valid.

This decomposition avoids making one graph serve simultaneously as a proof certificate, a search state, a corpus row, and a heuristic index.

Boundary and event definitions

Let a live search configuration be

$$S = (E, O, Q, M, U, P),$$

where E is the environment and source-prefix availability, O relevant options and scope, Q the ordered active-goal list, M the reachable term-metavariable declarations and assignments, U universe information, and P any relevant postponed or opaque obligations. This is an interface contract; exporting all of Lean’s operational state is not assumed feasible. Missing fields must be represented by coverage flags, not silently erased. Lean distinguishes term goals, shared unknowns, and universe metavariables. [R14, R15]

A goal capsule stores its local telescope, target, available local instances, identity, and content version. Metavariable types and local-definition values participate in dependency closure. Raw identifiers are for same-session provenance; model features use explicit structural references. Do not replace an unexported free variable by a common placeholder.

Define an observed boundary

$$\partial S = (Q, \text{Reach}(Q; M, U, P), \text{coverage}).$$

Reach follows references through goal targets, local types/values, metavariable declarations/assignments, and universes. Opaque constraints either join the affected component conservatively or disable factorization. A successful export is not automatically a complete export.

An event is

$$e_i = (b_i, S_i, t_i, o_i, S'_i, \eta_i),$$

with branch/transaction identifier b_i , tactic syntax t_i , outcome o_i , and evidence η_i . Outcomes distinguish unexecuted proposals, executed successes, errors, timeouts, and unknowns. Disposition separately distinguishes committed, speculative, and rolled-back events. A successful speculative alternative need not be committed.

Let ρ_i map active Lean goal identifiers injectively into persistent graph identities. Preserve this identity while incrementing a **capsule version** when an assignment changes the instantiated target or context. A removed active

goal is `no_longer_active` until assignment/reachability evidence determines whether it is solved, replaced, or suspended.

The durable history is append-only:

$$H_{i+1} = H_i \parallel [e_i].$$

The current graph is a materialized view:

$$G_{i+1} = \text{Apply}(G_i, \Delta_i).$$

Node insertions can be monotone, but statuses, active ports, and content versions are updates. Writing only $G_{i+1} = G_i \cup \Delta_i$ loses this distinction.

Evidence hierarchy

Use independent evidence labels rather than a single `verified` flag:

- `source_observed`: syntax, arguments, source span;
- `lean_transition_observed`: before/after state observed during an executed transition;
- `assignment_observed`: a particular assignment or expression reference was inspected;
- `derived_annotation`: an extractor rule inferred a relation;
- `kernel_replay_passed`: a complete reconstructed declaration passed checking under a recorded trust policy;
- `proposed`: no execution evidence.

An accepted tactic does not prove that a source-mentioned lemma was used. `constructor` does not certify sibling independence. A root proof with unresolved holes is not a complete theorem. Proof-term enrichment reveals actual references but does not automatically establish minimal dependencies or tactic commutation. [R14, R15]

The hashing limitation: an elementary but decisive result

Proposition 1 (refinement cannot enlarge duplicate classes). Let N be any state key and H any graph key. Define $K(x) = (N(x), H(x))$. Then

$$K(x) = K(y) \implies N(x) = N(y).$$

Therefore each K -equivalence class is a subset of an N -equivalence class.

Proof. Equality of ordered pairs entails equality of their first coordinates. $\square$

Consequence. A state-plus-graph exact key cannot, by itself, merge more states than that same state key. It can retain *more* distinctions. Any advantage must come from a different decision: scheduling, learned ranking, cross-context fragment reuse, or cheaper comparison. Appending graph history to the duplicate key can even undo useful transposition sharing.

This does not make graphs useless. It identifies which mechanism must be measured. A faithful state may contain all logical information while a graph remains a helpful computational representation. No information-theoretic superiority is claimed.

Conditional local independence, not tactic-name independence

For an explicitly restricted tactic semantics, let $R(t, S)$ and $W(t, S)$ denote the mutable cells read and written by a transition. If both transitions are applicable, allocation can be renamed, they perform no relevant external effects, and

$$W(t, S) \cap (R(u, S) \cup W(u, S)) = \emptyset,$$

$$W(u, S) \cap (R(t, S) \cup W(t, S)) = \emptyset,$$

with these footprints stable over the two executions, the usual disjoint-update argument yields commuting results up to fresh-identifier renaming.

This is a conditional frame argument, not a theorem about arbitrary Lean tactics. A graph of visible goal references is an approximation to these footprints. A shared witness can make a focused **exact** change another goal. Arbitrary metaprograms may inspect additional state. Consequently the production rule is: preserve observed identities, record every observed effect, and refuse uncertified factorization when coverage is insufficient. [R3, R5, R6, R14]

For **first**, **try**, and **repeat**, preserve nested transaction ancestry. An InfoTree node in a failed alternative must not be treated as a committed proof step solely because it is present in the tree. The current repository's handoff explicitly leaves these constructs as audit targets. [R4]

The recommended new algorithm: closed-fragment transport

Begin with a completed local proof, not an arbitrary open graph. Write

$$E; \Gamma \vdash p : T,$$

where p contains no unresolved expression or universe metavariables. Compute the dependency-closed local telescope Γ_p : free locals in p and T , plus all locals required by their types and retained local-definition values. Preserve order and instance/binder information. Abstract to a portable term over Γ_p ; do not simply erase unused-looking hypotheses from a printed state.

A candidate reuse at target $E'; \Gamma' \vdash T'$ supplies a typed context substitution θ and compatible environment. The proposed application is $\theta(p)$. The match is useful only if Lean accepts it at T' under the current declaration prefix and trust policy.

Proposition 2 (typed substitution, restricted setting). In a dependent type theory with the standard substitution rule, if $E; \Gamma \vdash p : T$ and $\theta : \Gamma \rightarrow \Gamma'$ is a well-typed context substitution, then

$$E; \Gamma' \vdash \theta(p) : \theta(T).$$

Proof sketch. Induct through the typing derivation; the variable case uses the substitution's typing assumptions, and binder cases extend the substitution with a fresh bound variable. Local definitions require compatibility with their values. If $\theta(T)$ is definitionally equal to T' , the conversion rule gives the desired judgment. $\square$

This is standard metatheory, not an original theorem claimed by the project. It motivates a concrete checker: build the candidate in Lean rather than trusting Python's matcher. Different environments require checking the used declarations, universe parameters, and permitted axioms; equal spelling of a constant is not sufficient.

Online procedure

For each actually solved local obligation:

```
inspect the resulting term in its saved Lean context;
reject unresolved holes, unexported locals, or unsupported obligations;
build its dependency-closed telescope and portable abstraction;
store term reference, interface, provenance, and replay evidence.
```

Before requesting another costly tactic batch:

```
query the ordinary typed cache and/or the boundary-graph index;
propose context substitutions using only the available prefix;
instantiate a candidate in a cloned current Lean state;
accept a reuse only if Lean accepts the application;
commit exactly the accepted branch and record all state effects;
otherwise restore state and continue the unchanged baseline search.
```

The index may be approximate. False matches are performance failures, not licenses to assert a proof. Keep final full-declaration replay. Typechecking the reused term protects theorem correctness; maintaining baseline fallbacks protects coverage but does not by itself guarantee equal success under a finite budget.

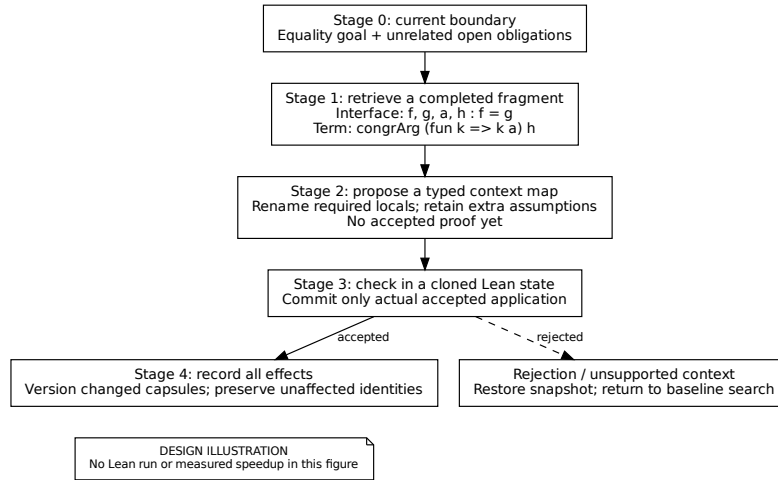


Figure 2: Stages of a proposed checked reuse. Retrieval and context matching remain proposals until Lean accepts the application.

Three instructive examples

A. Existing sibling transport is a regression, not a new speed result

From $h : p \wedge q$, use `constructor`, `exact h.right`, `exact h.left` to prove $q \wedge p$. The second tactic must preserve the identity of the remaining p goal. Both the user artifact and the current research line already target this case. It tests extraction, not an incremental graph-search advantage.

B. Context-sensitive reuse: the actual new hypothesis

Suppose a completed local term proves $f a = g a$ from $h : f = g$. In another branch, the context includes extra unrelated assumptions and differently named f, g, a, h . The whole context key can differ even though the fragment `congrArg (fun k => k a) h` transports. A dependency-indexed cache can propose the fragment using only its required telescope; Lean checks the application.

Graph stages are: an open equality port; a retrieved fragment with unbound interface parameters; a proposed typed substitution; a successful local application; and the unchanged external frontier. This could avoid re-searching the local argument. However an ordinary proof-term cache could do exactly the same, so it is a mandatory baseline. This is a design example, not a measured result.

C. Coupled witnesses: why local successes cannot be combined freely

For an open construction of $\exists n : \mathbb{N}, n = 0 \wedge n = 1$, a shared witness cell w links the residual goals $w = 0$ and $w = 1$. One isolated branch can succeed by setting $w := 0$, another by setting $w := 1$. Their union is inconsistent. A reuse packet for an open fragment must include assignment constraints and jointly check composition; a cache that stores only “this goal succeeded” is insufficient. This is why closed-fragment reuse precedes open-fragment reuse.

Interface and integration

Keep the frozen v0.2 graph dataset unchanged. Add `joint-sidecar/0.1.0` records keyed by source commit, declaration locator, extraction session, InfoTree/event path, and content hash. The existing exporter does not provide rich boundary state, so its imported sidecars truthfully contain `state_before: null`, `state_after: null`, and `boundary_coverage: not_exported`. [R7, R8, R17]

Use a tagged JSON expression codec rather than renaming over serialized strings. Preserve literals and alias structure; handle missing declarations as unknown. Use full digests for indexing and retain canonical bytes for

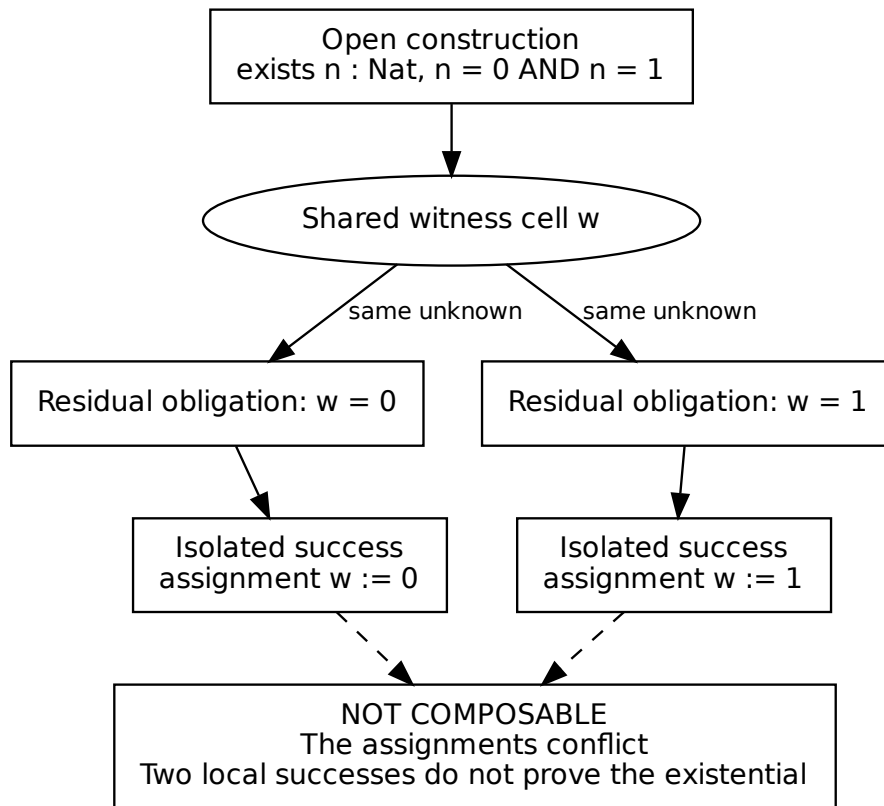


Figure 3: Why a shared witness must remain in the interface of an open fragment. This is a logical counterexample to naive composition, not an executed Lean trace.

collision resolution. Neither a digest nor a transport map is a proof of Lean-state equivalence.

The reference code in this package demonstrates those representation properties in a finite model. It deliberately rejects incomplete fixture coverage. It does not supply a Lean exporter, a complete operational boundary, a proof-term matcher, or kernel replay.

Scope of novelty

Existing work supplies the MLL theory, hypergraph search, Lean interfaces, factorized state reuse, and expression graphs. [R1, R10–R16] This proposal adapts those ideas into an auditable composition of state snapshots, event effects, portable proof fragments, and checked reuse. The original deliverable here is the integration design and its adversarial regression model, not a claim to have invented substitution, proof graphs, metavariable coupling, or memoization.

Tactician’s Web already connects Coq definitions, proof terms, tactics, and states in a semantic graph with an interactive interface; Graph2Tac and Tactician’s online learning show that reuse of nearby formal knowledge is also an established research theme. [R18, R19] These precedents further rule out claiming generic semantic proof graphs or online reuse as new. The proposed contribution must be its precise Lean-side contract, independent evidence, and measured incremental behavior.

Only experiments can establish whether the joint representation adds useful value. If a plain typed cache matches graph-indexed reuse, keep the cache and report that the graph-specific hypothesis failed.

Experiment and implementation plan

Registration status: DRAFT. This document is not a preregistration. Freeze code, corpus, hypotheses, exclusions, costs, and analysis in a new commit before collecting new outcome data. Do not edit existing registered experiments or reinterpret their decisions. [R3, R4]

Work package 1: reproduce and audit the actual baseline

Work in a local branch of `proof-graphs`, pinned initially at `8010d137af5af5040fcbcd9e98f910b01162263e`. Keep its actual dependency lock; do not substitute the newer `proofnet-ir` main for a pinned dependency. Compare that pin with the separately inspected ProofNet-IR head and document the distinction.

Run the documented build and committed-artifact checks. Distinguish recomputing summaries from running new inference. Check a small set of fresh Lean fixtures before a broad export. Select fixtures for nested `have`, `calc`, `simp` ... `using`, `first`, `try`, `repeat`, shared witnesses, metavariable-dependent local types, and universe sharing. A human should independently reconstruct at least the failure-combinator cases before viewing extractor output. [R4]

Acceptance: recorded commands and logs; untouched frozen outputs; a reproducible failing test for each confirmed defect, or an explicit refutation of the suspected defect. No mass source transformation or model training is required.

Work package 2: a fail-closed typed boundary sidecar

Extend the existing Lean exporter and REPL rather than introducing another session driver. Required information is the active goal list, persistent raw IDs, versioned capsules, tagged expressions and levels, local telescope, reachable metavariable declarations/assignments, environment/prefix fingerprint, branch identity, and dependency-coverage flags.

When complete coverage is unavailable, keep the entire affected state/group and avoid hard equality merges based on lossy printed fallback. Lack of factoring is an acceptable conservative result. Querying the same snapshot twice must not change observable proof behavior.

Acceptance: all adversarial fixtures pass; no literal modification or placeholder identity merges; same raw goal may change its capsule version; failures do not mutate the committed graph. Every exported row validates against one current schema. Recorded gold evidence is attached only to actual full replay, not to a tactic name.

Work package 3: closed-fragment cache with a graph ablation

Use only completed, hole-free local terms. Start with same-environment reuse under typed renaming and weakening. Do not start with arbitrary open-subproof unification or a learned GNN.

Arms:

Arm	Purpose
A0: faithful whole-state search	Existing baseline, same correctness repairs as other arms
A1: faithful coupled-group search	Existing graph baseline
A2: A0 plus ordinary typed closed-term cache	Does reuse help without graph-specific indexing?
A3: A0 plus boundary/graph-indexed cache	Does your representation improve retrieval or net cost?

A2 and A3 receive the same proof-fragment inventory, trust policy, verification budget, and candidate proposer. A3 cannot quietly receive more theorems, future dependencies, or a stronger model. Repair keying in all arms; do not credit fixing an old baseline bug as a graph-specific learning benefit.

Primary mechanism metrics: eligible repeated fragments, retrieval attempts, successful Lean-checked reuse, rejected transports, avoided model requests, avoided tactic executions, export/match/recheck costs. Primary task metric: theorem success under an explicitly fixed resource budget. Report all denominators and setup failures.

A graph feature could improve retrieval order without supporting any extra exact merges. Make that the experimental distinction rather than comparing a weak string cache to a rich graph cache.

Work package 4: held-out mathematical pilot

First use 20–50 small and medium proof fragments with genuine repeated subarguments and adversarial negatives; choose the final count before the run. After extraction and transport are stable, select a separate held-out module set in analysis, algebra, or category theory. Curate by syntactic constructs and available build resources, not by whether the new algorithm wins.

For condensed mathematics, use an actually buildable, pinned Lean project and identify its Lean generation. The historical Liquid Tensor corpus and a modern Lean 4 Mathlib development are not interchangeable. No theorem from that area was extracted in this review. The milestone is one correct traced module fragment, not a diagram labelled frontier-scale.

Controls that matter

Leakage. Split by module/theorem family and dependency availability. Exclude the target declaration itself and later declarations from the proving environment. Store full proofs for audit separately from policy inputs. Do not compute input graph features from future proof steps.

Common draws. For controlled search comparison, prerecord or deterministically index candidate draws by exact request content, seed, and occurrence. Report marginal candidate costs rather than which arm happened to populate the shared cache. A policy conditioned on a different graph prompt cannot be called the same draw experiment without describing the changed request.

Runtime. Run isolated fresh-process cold-cache trials, randomized by arm order, under an explicit warmup policy. Include extraction, serialization, canonicalization, matching, replay, and model time. The corrected search-v0.8 cost discussion makes this a required control. [R3]

Statistics. A theorem repeated under seeds is a clustered observation. Use paired per-task comparisons and module-aware uncertainty estimates. Predefine multiple-comparison handling. Nonsignificance is not equivalence; a claim of non-inferiority needs a chosen margin and an appropriate design.

Go / no-go gates

Proceed to a trained graph model only after there is evidence of a useful signal beyond a plain cache. If eligible reuse is rare, change corpus or conclude limited applicability before optimizing graph encoders. If replay overhead

exceeds avoided work, report the cost failure. If A2 and A3 perform alike, remove graph-specific claims and retain the useful semantic tooling.

The immediate decision need not depend on a positive ML result: a carefully audited extractor and falsifiable account of where abstraction fails are legitimate outputs, though their publication fit depends on the resulting evidence and venue.

Contribution and collaboration proposal

Your part, stated without inflating it

Your last available implementation is smaller and less mature than your friend’s current repositories. Its strengths are the original problem framing, the focus on partial boundaries rather than finished proofs alone, and the insistence on extracting faithful intermediate objects. Those strengths become a distinct contribution only when they are turned into new checks, semantics, or demonstrated reuse—not when renamed as another graph abstraction. [R1–R8, U1]

This assessment compares artifacts. It cannot reconstruct the history of your joint conversations, the origin of shared ideas, or individual authorship. Agree on contribution credit directly and keep a written record of work performed.

Proposed ownership, subject to your agreement

Workstream	Suggested lead	Concrete output
Restricted MLL theory and certified IR	Friend / existing maintainer	Stable theorem interface; no unsupported transfer to Lean graphs
Existing extraction, search, and frozen results	Friend / existing maintainer	Baseline behavior, reproducibility support, review of changes
Independent semantic audit	You	Hand reconstructions, adversarial Lean fixtures, reproducible bug reports
Typed boundary / event interface	You, jointly reviewed	Versioned schema, exporter sidecar, provenance, transaction checks
Checked fragment transport	You, jointly reviewed	Closed-fragment cache, context mapping, replay evidence
Experimental design and interpretation	Both	Frozen protocol, strong baselines, negative results retained
Paper and release	Both	Separate formal claims, engineering guarantees, empirical conclusions

Do not divide the project as “he does code; I do ideas.” Own a module with measurable acceptance criteria. Conversely, a correctness audit is not inferior to new features: it determines whether the features support the intended scientific claims.

Proposed sequence of pull requests

PR 1: audit only. New adversarial fixtures and source-pinned review. No baseline behavior change. Establish which suspicious cases are real defects.

PR 2: typed boundary sidecar. Additive schema and serializer, identity-version separation, conservative unknown handling. Keep old datasets and frozen experiments intact.

PR 3: checked closed-fragment cache. Baseline cache and graph-index variant using the same inventory. No learned model yet.

PR 4: registered experiment and report. Collected only after the protocol commit. Explain null or negative results as clearly as positive ones.

No automatic push to main. Work on a local feature branch, request review, and resolve concurrent-edit ownership. The final ZIP is a planning/reference package, not a modification of either repository.

Suggested message to your friend

“I read the newer results and think I should stop duplicating the extractor and goal-grouping work you already have. I’d like to own the semantic audit and typed boundary/transport layer, then test whether checked reuse of local proof fragments adds anything beyond your structural-state and group baselines. I will start with the failure-combinator and identity tests, keep your frozen experiments intact, and include a plain proof-term cache so we can isolate whether the graph really helps. Let’s agree on that scope and review the interfaces together before implementation.”

What was and was not executed

Actually executed here

The supplied user v0.7.1 archive was read programmatically and checked against its shipped JSON Schema. `evidence/USER_V071_AUDIT.json` records the counts and mismatch examples. This is a fresh artifact audit, not a fresh Lean run.

The package’s Python reference tests were run with `python -m unittest discover -s tests -v`. They cover typed renaming, literal preservation, metavariable aliasing, shared local identity across capsules, transitive and univariate dependencies in a finite model, frontier persistence, and schema guards. `evidence/REFERENCE_TEST_LOG.txt` is the actual output.

The literal regression reproduces the specific regex-renumbering mechanism observed in the pinned `scripts/goal_identity.py` [R5]. It does not import and execute the entire remote repository, and its synthetic inputs are not evidence of an actual erroneous Mathlib proof.

Not executed here

No local `lake build`, Lean proof elaboration, friend-repository end-to-end reproduction, full 7,285-row dataset conversion, new model call, or frontier theorem trace was performed. Lean/Lake was unavailable; container GitHub cloning failed at DNS resolution, while selected files were successfully read through the connected GitHub tool. No repository was modified remotely.

`examples/BoundaryRegression.lean` is uncompiled fixture source. The JSON examples are explicitly synthetic design fixtures. Their IDs name fixtures, not invented Lean metavariables. The offline adapter was tested on a synthetic row matching the inspected format, not the full downloaded dataset.

Interpretation limits

The Python representation is not a Lean typechecker and does not decide operational state equivalence. It deliberately rejects missing declared dependencies, but its finite-model closure does not establish complete coverage of Lean’s postponed constraints, plugins, transparency behavior, elaborator state, or external effects. A matching fingerprint licenses no hard pruning. The sidecar schema enforces that limitation.

A passed JSON schema checks structure, not provenance truth. A claimed log reference still requires checking its contents. A passed local term application is not a full-declaration replay. A full checked theorem does not prove that every extractor annotation is correct or that a pruning rule preserves search completeness.

Source register

Reviewed 30 September 2026. GitHub links are pinned to the exact inspected commit. The Lean reference is a moving documentation URL; actual integration must inspect the pinned Lean source. R-numbers below are used throughout the reports. U1 is a local artifact, not a public publication.

- [R1] **ProofNet-IR: current status and restricted MLL guarantees.** <https://github.com/fushanbobfan/proofnet-ir/blob/261f2db26aac842534f4306da0f45d8c5e9b1c05/docs/current-status.md>
- [R2] **proof-graphs: current scope and dataset.** <https://github.com/fushanbobfan/proof-graphs/blob/8010d137af5af5040fcbcd9e98f910b01162263e/README.md>
- [R3] **proof-graphs: states, goals, and coupled groups, search-v0.8.** <https://github.com/fushanbobfan/proof-graphs/blob/8010d137af5af5040fcbcd9e98f910b01162263e/experiments/search-v0.8/README.md>
- [R4] **proof-graphs: handoff, verification boundaries, and audit priorities.** <https://github.com/fushanbobfan/proof-graphs/blob/8010d137af5af5040fcbcd9e98f910b01162263e/HANDOFF.md>
- [R5] **proof-graphs: structural goal identity implementation.** https://github.com/fushanbobfan/proof-graphs/blob/8010d137af5af5040fcbcd9e98f910b01162263e/scripts/goal_identity.py
- [R6] **proof-graphs: structural-key whole-state, goal, and group searches.** https://github.com/fushanbobfan/proof-graphs/blob/8010d137af5af5040fcbcd9e98f910b01162263e/scripts/search_faithful.py
- [R7] **proof-graphs: Lean InfoTree extractor.** <https://github.com/fushanbobfan/proof-graphs/blob/8010d137af5af5040fcbcd9e98f910b01162263e/ProofGraphs/Extract.lean>
- [R8] **proof-graphs: corrected graph derivation and dataset generation.** https://github.com/fushanbobfan/proof-graphs/blob/8010d137af5af5040fcbcd9e98f910b01162263e/scripts/run_recount.py
- [R9] **ProofNet-IR: representation study with and without thinking, v0.3.** <https://github.com/fushanbobfan/proofnet-ir/blob/261f2db26aac842534f4306da0f45d8c5e9b1c05/experiments/representation-v0.3/README.md>
- [R10] Kripner, Sustr, and Straka (2025), **LeanTree: Accelerating White-Box Proof Search with Factorized States in Lean 4.** <https://arxiv.org/abs/2507.14722v1>
- [R11] Lample et al. (2022), **HyperTree Proof Search for Neural Theorem Proving.** <https://arxiv.org/abs/2205.11491v1>
- [R12] Aniva, Sun, Miranda, Barrett, and Koyejo (2025 revision), **Pantograph.** <https://arxiv.org/abs/2410.16429v2>
- [R13] Aniva, Oikawa, Dill, and Barrett (2026), **Nazrin: An Atomic Neural Proof Automation Tactic in Lean 4.** <https://arxiv.org/abs/2602.18767v3>
- [R14] **Lean language reference: reading proof states.** <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Reading-Proof-States/>
- [R15] **Lean language reference: tactic proofs.** <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/>
- [R16] Yang et al. (2023), **LeanDojo: Theorem Proving with Retrieval-Augmented Language Models.** <https://arxiv.org/abs/2306.15626>
- [R17] **proof-graphs: original dataset row format; use R8 for corrected v0.2 generation.** https://github.com/fushanbobfan/proof-graphs/blob/8010d137af5af5040fcbcd9e98f910b01162263e/scripts/export_graphs.py
- [U1] **User-supplied Boundary-Lens v0.7.1 archive; fresh Python audit in this package.** [../evidence/USER_V071_AUDIT.json](https://evidence.USER_V071_AUDIT.json)
- The repository reports are first-party evidence, not independently replicated results in this review. Selected code paths were inspected through the connected GitHub reader. Container cloning was blocked by DNS and Lean/Lake was unavailable. The review therefore does not certify either full repository build or all experimental conclusions.
- [R18] Blaauwbroek et al. (2024), **Graph2Tac: Online Representation Learning of Formal Math Concepts, ICML / PMLR 235.** <https://proceedings.mlr.press/v235/blaaubroek24a.html>
- [R19] Blaauwbroek (2024), **The Tactician’s Web of Large-Scale Formal Knowledge.** <https://arxiv.org/abs/2401.02950v2>